

claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf\_c287c071-8e9\agent-a76da7693ae61b81c.jsonl`
- SHA-256: `722a3cc5fa53ed0ef921e7624443a42f3eca682e90d3b02569db6a4d3b57a614`
- Source modified: `2026-07-07T14:54:11+00:00`
- Imported at: `2026-07-08T16:00:01+00:00`
- project: `wf\_c287c071-8e9`
- session_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`

Transcript

1. user (2026-07-07T14:50:32.862Z)

You are reviewing an uncommitted change in a git worktree at:
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval
Branch: fix/worker-skip-revalidation-on-dispatch (based on origin/master).

THE BUG BEING FIXED: On the cloud-serving path, the control-plane (backend/orchestration.py) runs validators and ONLY dispatches to the Cloud Run worker on PASS. The worker (backend/worker/__main__.py cmd_infer) then REDUNDANTLY re-ran validation with its OWN config (different min_blur_threshold), rejecting a clip the control-plane already passed. Live failure 2026-07-07: passed at min_blur_threshold=8.0 on the control-plane, worker rejected at its default 20.0 ? GPU dispatch succeeded but the CUJ failed with worker rc=2.

THE FIX (skip-on-dispatch): a new ComputeJobSpec.skip_validation bool (default False) ? backend/compute/cloud_run.py emits a "--skip-validation" worker CLI flag when set ? backend/worker/__main__.py cmd_infer honors it (skips validator_registry.run_all_with_context, sets val_results=[], val_context={}) ? backend/orchestration.py::_maybe_dispatch_remote sets skip_validation=True on every dispatch. The direct "worker infer" CLI (no control-plane) still validates by default. New tests in tests/test_worker.py and tests/test_api_cloud_dispatch.py.

Inspect the ACTUAL code ? do NOT trust this summary. Run: cd "C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval" && git --no-pager diff and read the surrounding functions (not just the diff hunks). The full test suite already passes (1779 passed), ruff + mypy are clean, coverage 85.42%. So do NOT report style nits or things the gate already covers. Hunt for REAL correctness/rollout/consumer defects the tests would miss. Only report a finding you can tie to a concrete failure scenario in THIS repo's code.

A reviewer (lens: consumers) raised this finding about the change. ADVERSARIALLY verify it against the REAL code in C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval (read the files; run git grep / git diff). Default to REFUTED unless you can demonstrate the concrete failure in THIS repo's code. If real, give the minimal fix.

FINDING:
severity=major category=rollout-skew
file=backend/orchestration.py line=582
summary=Control-plane forwards the brand-new hard argparse flag --skip-validation on EVERY cloud dispatch unconditionally; a control-plane-ahead-of-worker deploy makes every dispatch argparse-fail (rc 2) on the old worker image.
detail=_maybe_dispatch_remote sets skip_validation=True with no guard, and cloud_run.py run_infer emits it as the argparse store_true flag "--skip-validation" (worker/__main__.py build_parser, lines 743-750). The control-plane service (gcloud run services replace / deploy

rally-control-plane) and the worker Cloud Run Job (gcloud run jobs update rally-worker) are the SAME image but SEPARATE deploy targets rolled by separate commands (deploy/cloudrun/README.md §1-3; #510 runlog shows 'control-plane redeployed' as its own event). Any worker image predating this change has no --skip-validation in its parser, so build_parser().parse_args([..., '--skip-validation', ...]) raises SystemExit(2) ('unrecognized arguments') before cmd_infer runs. The repo already documents this exact hazard for decode_backend (README lines 76-78: 'a new control plane forwarding nvdec to an old worker image argparse-fails (rc 2). Roll the worker image and the control plane together'), but (a) that 'roll together' note was NOT extended to mention --skip-validation, and (b) unlike decode_backend ? which is forwarded only when an operator opts into the nvdec preset and has a WASB_DECODE_BACKEND env escape hatch ? --skip-validation is forwarded on 100% of cloud dispatches with no config gate and no env override, so the skew takes down ALL cloud serving, not just an opted-in subset. Mitigation options: extend the README 'roll together' warning to skip_validation, and/or forward via the soft '--set' mechanism (which an old worker degrades on rather than hard-argparse-failing) or gate the forward behind a worker-image capability check.

failure_scenario=Operator pushes new rally-worker:latest and runs `gcloud run deploy rally-control-plane --image ../rally-worker:latest` to roll the control plane, but has not yet run `gcloud run jobs update rally-worker --image ...`, so the Job still runs the prior image (a routine partial rollout; the runbook lists these as separate steps). A user submits a gs://video: the new control plane validates PASS and dispatches the Cloud Run Job with argv containing `--skip-validation`. The old worker image's argparse rejects the unknown flag with SystemExit(2) before cmd_infer executes, so no done-marker is written; the control plane bucket-polls to PolicyTimeout / returns status='failed'. Every cloud dispatch fails until the worker Job image is also rolled ? a worse blast radius than the decode_backend hazard the README already warns about, because this flag is forwarded unconditionally on every dispatch.

2. assistant (2026-07-07T14:50:38.615Z)

I'll investigate this finding against the real code. Let me start by examining the diff and relevant files.

3. user (2026-07-07T14:50:40.846Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
     frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
+    plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
+    and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
+    and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
+    a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
+    plane)
+    # validating exactly as before.
+    skip_validation: bool = False
```

```
@dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
     ]
     if spec.segmenter:
         argv += ["--segmenter", spec.segmenter]
```

```

+         if spec.skip_validation:
+             # The control-plane already validated + only dispatches on PASS ? tell the worker
not to
+             # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
+             argv.append("--skip-validation")
+             for kv in (*spec.config_overrides, *self._container_overrides):
+                 argv += ["--set", kv]

diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
    prefetch = getattr(wasb_cfg, "prefetch_batches", None)
    if prefetch:
        overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
+
+ # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
+ # gate ? it already ran the validators with its resolved config just above (only a PASS
reaches
+ # here; a failure returned "failed" before dispatch). A worker re-run would decode the
video a
+ # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
+ # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+ # then worker-rejected at its default 20.0 after a successful GPU dispatch).
    spec = ComputeJobSpec(
        video_uri=req.video_path,
        out_uri=out_root,
        segmenter=seg_name,
        config_overrides=tuple(overrides),
+
+     skip_validation=True,
    )
    try:
        # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    out_uri=args.out_uri,
    segmenter=args.segmenter,
    config_overrides=tuple(args.set or ()),
+
+     skip_validation=bool(getattr(args, "skip_validation", False)),
    )
    try:
        result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

-
- # 3. VALIDATE (same registry as the main CLI).
- val_results, val_context = validator_registry.run_all_with_context(
-     local_video, config
- )
+
+ # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+ # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+ # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+ # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config

```

```

+ # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+ # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+ # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+ # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+ # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+ if getattr(args, "skip_validation", False):
+     print(
+         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+         "gate and already validated this input."
+     )
+     val_results, val_context = [], {}
+ else:
+     val_results, val_context = validator_registry.run_all_with_context(
+         local_video, config
+     )
+     failures = [r for r in val_results if not r.passed]
+     db = Database(os.path.join(scratch, config.output.db_name))
+     db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
+         metavar="k=v",
+         help="config override, e.g. --set indexing.skip_ai_handoff=true",
+     )
+     pi.add_argument(
+         "--skip-validation",
+         action="store_true",
+         help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+         "validators with its resolved config and only dispatches on PASS, so re-validating here
is "
+         "redundant, decodes the video again, and can contradict the gate. Default-OFF: the
direct "
+         "`worker infer` CLI still validates.",
+     )
+     pi.set_defaults(func=cmd_infer)

    pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
    assert "indexing.wasb.prefetch_batches" not in joined

+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
wiring."""
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+    )

```

```

+     assert "--skip-validation" in fake.calls[0]
+
+
+ def test_cloud_failure_marks_failed_no_rows(cloud_env):
+     db, _bucket, monkeypatch = cloud_env
+     # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
     assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]

+def test_parser_infer_skip_validation_flag():
+    base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+    # Default-OFF: the direct `worker infer` CLI still validates.
+    assert wmain.build_parser().parse_args(base).skip_validation is False
+    # The dispatcher passes it to disable the redundant worker re-validation.
+    assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
+
+
+ # ----- cmd_infer (local, mocked)

@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
     assert marker["returncode"] == 2 and marker["status"] == "failed"

+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+    """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
worker's
+    validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
+    control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+    run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
a
+    status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
2)."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+    def _boom(*a, **k): # the validators must never be reached on the skip path
+        raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+    video = tmp_path / "in.mp4"
+    video.write_bytes(b"FAKE")
+    out = tmp_path / "bucket"
+
+    rc = wmain.main(
+        [
+            "infer",
+            "--video-uri",
+            str(video),
+            "--out-uri",
+            str(out),
+            "--scratch",
+            str(tmp_path / "s"),
+            "--segmenter",
+            "wasb_hybrid",
+            "--skip-validation",
+        ]
+    )

```

```
+ assert rc == 0 # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+ report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+ assert report["status"] == "success" and report["segment_count"] == 2
+
+
# --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---
```

4. assistant (2026-07-07T14:50:46.295Z)

Let me examine the referenced files: the README hazard note, deploy docs, and how decode_backend is forwarded.

5. user (2026-07-07T14:50:50.104Z)

Exit code 2

README.md:305:The AI provider used by `yolo\_hybrid` and `gemini` is controlled by `ai\_provider` + `ai\_model` at the top level of `config.json`. Today only `gemini` is registered; new providers can be added under `backend/providers/`.

---README lines 70-90---

The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract keyframes, and stitch clips.

```
* **On Windows (PowerShell as Admin)**:
  ``powershell
  winget install Gyan.FFmpeg
  ```
 Restart your terminal afterwards to register the path changes.
```

#### 2. Install the package

The project is packaged with PEP 621 metadata in `pyproject.toml` (hatchling build backend). Install it editable:

```
```bash
```

Runtime deps only:

```
pip install -e .
```

Plus test + lint/type tooling (pytest, pytest-cov, httpx, ruff, mypy):

```
pip install -e .[dev]
```

```
```
```

This registers the `indexer` console entry point (so `indexer --server`, `indexer --help`, etc. work) and puts the `backend` and `training` packages on your path.

> **PyTorch** is installed separately. **torch**/**torchvision** are intentionally

---find README---

deploy/cloudrun/README.md

README.md

#### 6. user (2026-07-07T14:50:51.698Z)

```
backend/config/models.py:122: # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident"
moves decode+resize+normalize
backend/config/models.py:131: decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
backend/config/models.py:134: def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
backend/config/models.py:138: if self.decode_backend == "nvdec_resident" and self.device
not in ("cuda", "auto"):
backend/config/models.py:140: f"indexing.wasb.decode_backend='nvdec_resident'
requires device 'cuda' or "
backend/config/presets.py:63: # out + canaried (DEPLOY_REPORT_cloud-serving-gpu-
resident-nvdec_2026-07-07). This makes
backend/config/presets.py:64: # nvdec the code-level serving default so it survives
an image rebuild; the worker's
```

```

backend/config/presets.py:65: # WASB_DECODE_BACKEND env was the reversible rollout
lever. cuda-only (validator-enforced).
backend/config/presets.py:69: "decode_backend": "nvdec_resident",
backend/orchestration.py:556: # without forwarding, its decode_backend + GPU-feed tuning
silently fall back to the model
backend/orchestration.py:557: # defaults (cpu, batch 8) regardless of what the control-plane
resolved. decode_backend is
backend/orchestration.py:558: # cuda-only (validator-enforced) and requires a worker image
that understands --decode-backend
backend/orchestration.py:562: decode_backend = getattr(wasb_cfg, "decode_backend", None)
backend/orchestration.py:563: if decode_backend:
backend/orchestration.py:564:
overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
backend/pipeline/detectors/native_wasb_runner.py:63:# Decode backends wasb_infer.py understands
(#506). Mirrored by its --decode-backend choices;
backend/pipeline/detectors/native_wasb_runner.py:65:_VALID_DECODE_BACKENDS = ("cpu",
"nvdec_resident")
backend/pipeline/detectors/native_wasb_runner.py:103: # GPU-resident NVDEC decode (#506,
indexing.wasb.decode_backend / WASB_DECODE_BACKEND).
backend/pipeline/detectors/native_wasb_runner.py:105: # byte-identical. "nvdec_resident" =
decode+resize+normalize on-GPU (torch-2.x env only).
backend/pipeline/detectors/native_wasb_runner.py:106: decode_backend: str = "cpu"
backend/pipeline/detectors/native_wasb_runner.py:132:
decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
backend/pipeline/detectors/native_wasb_runner.py:221: if self.cfg.decode_backend not in
_VALID_DECODE_BACKENDS:
backend/pipeline/detectors/native_wasb_runner.py:223: f"unknown decode_backend
{self.cfg.decode_backend!r} ? valid: "
backend/pipeline/detectors/native_wasb_runner.py:257: if self.cfg.decode_backend ==
"nvdec_resident" and ctx.effective != "cuda":
backend/pipeline/detectors/native_wasb_runner.py:259:
f"decode_backend=nvdec_resident requires CUDA, but the effective device is "
backend/pipeline/detectors/native_wasb_runner.py:262: if self.cfg.decode_backend ==
"nvdec_resident":
backend/pipeline/detectors/native_wasb_runner.py:269:
f"decode_backend=nvdec_resident but `{_PYNVC_PROBE_CODE}` failed in "
backend/pipeline/detectors/native_wasb_runner.py:306: if self.cfg.decode_backend ==
"nvdec_resident":
backend/pipeline/detectors/native_wasb_runner.py:310: argv += ["--decode-backend",
self.cfg.decode_backend]
backend/pipeline/detectors/native_wasb_runner.py:321: if self.cfg.decode_backend ==
"nvdec_resident" and device != "cuda":
backend/pipeline/detectors/native_wasb_runner.py:325:
f"decode_backend=nvdec_resident requires CUDA, but the effective device is "
backend/pipeline/detectors/native_wasb_runner.py:344: if self.cfg.prefetch_batches and
self.cfg.decode_backend != "nvdec_resident":
backend/pipeline/detectors/wasb_infer.py:37:``--decode-backend nvdec_resident`` (default
``cpu``) moves decode + resize + normalize
backend/pipeline/detectors/wasb_infer.py:66:# --stream-video); "nvdec_resident" = GPU-resident
NVDEC decode + affine grid_sample
backend/pipeline/detectors/wasb_infer.py:68:DECODE_BACKENDS = ("cpu", "nvdec_resident")
backend/pipeline/detectors/wasb_infer.py:215: decode_backend: str = "cpu",
backend/pipeline/detectors/wasb_infer.py:223: ``decode_backend`` follows the same pattern
(#506): pre-existing caches (no field)
backend/pipeline/detectors/wasb_infer.py:224: are ``cpu``-decoded, so they stay compatible
with cpu runs ? but an ``nvdec_resident``
backend/pipeline/detectors/wasb_infer.py:244: and manifest.get("decode_backend", "cpu")
== decode_backend
backend/pipeline/detectors/wasb_infer.py:620: decode_backend="cpu", # run() IS the cpu
decode path (disk frames / cv2 stream)
backend/pipeline/detectors/wasb_infer.py:641: "decode_backend": "cpu",
backend/pipeline/detectors/wasb_infer.py:992: ""On-GPU frame source for the
``nvdec_resident`` detector pass (#506).
backend/pipeline/detectors/wasb_infer.py:1015: f"decode_backend=nvdec_resident
needs PyNvVideoCodec with RGB output "

```

```

backend/pipeline/detectors/wasb_infer.py:1017: f"/ gpu_setup.sh stage them). Fall
back to decode_backend=cpu."
backend/pipeline/detectors/wasb_infer.py:1062:def run_video_nvdec_resident(
backend/pipeline/detectors/wasb_infer.py:1082: ``decode_backend`` and never shared with cpu-
decoded runs. Same resumable
backend/pipeline/detectors/wasb_infer.py:1098: raise
SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
backend/pipeline/detectors/wasb_infer.py:1113: f"nvdec_resident: {total} frames
({fe.orig_w}x{fe.orig_h}), "
backend/pipeline/detectors/wasb_infer.py:1117: # ---- cache setup / resume decision
(mirrors run(); keyed nvdec_resident) --- #
backend/pipeline/detectors/wasb_infer.py:1132: decode_backend="nvdec_resident",

```

## 7. assistant (2026-07-07T14:51:04.913Z)

Let me search for the exact README hazard text the finding cites, and check the deploy docs.

## 8. user (2026-07-07T14:51:09.720Z)

```

=== search argparse-fail / rc 2 / roll together in all docs ===
deploy/cloudrun/README.md:78:control plane forwarding nvdec to an old worker image argparse-
fails (rc 2).
deploy/cloudrun/README.md:101:boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even
pulled.
deploy/cloudrun/README.md:149:- A failed Job (validation rc 2 / segmenter rc 3) now writes a
failure marker carrying the
docs/COMPUTE_DECOUPLED_SERVING/README.md:169:- 2026-06-21 ? M5 code landed
([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)):
`backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-
cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner
healthcheck stays the authority), wired into the server (`enforce_startup_or_exit` ? exit 1) +
the worker (`cmd_infer`/`cmd_train` ? rc 2), default-OFF (probe gated behind an active profile ?
the worker stays torch-free at `profile=""`). L4 profile-parity test
(`tests/test_profile_parity.py`): same stub trajectory ? byte-identical windows + segment ranges
across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review
folded (`wf_34671ac1`). Suite green, ruff+mypy clean. ?? M5 ?? the real-GPU truly-local runlog
(L5) is owner-side (template pre-staged in `runlogs/`).
=== README lines 72-82 ===
deploy/cloudrun/README.md:76:? Forwarding `decode_backend=nvdec_resident` requires a worker
image that understands
deploy/cloudrun/README.md:77:``--decode-backend` (? #507). Roll the worker image and the
control plane together ? a new
deploy/cloudrun/README.md:78:control plane forwarding nvdec to an old worker image argparse-
fails (rc 2).
docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md:366:A11 ? WASB-C3 provenance (weights NOT commercially
cleared). The worker image itself carries the warning: "the pretrained badminton weights are
academic-data-trained ? provenance NOT cleared for commercial sharing? Resolve WASB-C3 before
any public, non-owner serving" (`deploy/cloudrun/Dockerfile:14-16`). The tracker now closes A11
for the limited product alpha by explicitly deferring C3 with guardrails, and #486 tracks the
provenance/counsel or cleared-detector path required before public/model launch. Consequences:
docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md:388:The control plane and worker run as the project
default compute SA (`492532266377-compute@developer.gserviceaccount.com`). What is known about
its privileges: it holds self-`roles/iam.serviceAccountTokenCreator` and uses it for keyless
signBlob (e2e runlog §4, repo-cited), it received `roles/secretmanager.secretAccessor` on
`gemini-api-key` this session (§5.4), and per session records it holds project-wide
roles/editor [unverified against live IAM ? one `gcloud projects get-iam-policy` check].
That SA backs a service carrying an allUsers` run.invoker binding (required for CF-proxied
traffic; the CF-JWT check is the auth). Net: an application-level RCE or SSRF in the CP would
inherit near-project-wide write access [analysis]`.
docs/COMPUTE_DECOUPLED_SERVING/README.md:121:| M6 | Cloud Run GPU dispatch shim (control
plane ? job via storage ? re-claim) + containerized worker image (ffmpeg+CUDA+WASB+fonts);
mocked-dispatch tests | M5 | ? owner (GCP spend) | ? |
[#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287) (shim+image+mocked ?;
real build/run = M7 ? owner) |
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_2026-07-04.md:10:- Worker

```



image builds + pushes (`asia-southeast1/rally`); the M6 `CloudRunCompute` shim dispatches the docs/COMPUTE\_DECOUPLED\_SERVING/runlogs/DEPLOY\_REPORT\_cloud-serving\_a1-e2e\_2026-07-04.md:18:| Worker image | `?/rally/rally-worker:latest` ? build 1 `sha256:ed2fe448?` (195932f), build 2 `sha256:31475f27?` (3c97117); the Job re-pinned via `gcloud run jobs update` after each build | docs/archives/decisions/DECISIONS.md:712:- **Cost/new surface:** a new `deployment`+`compute\_target` config surface, a Cloud Run **dispatch shim**, a **containerized GPU+ffmpeg worker image**, a **cost-ledger migration**, and **edge auth** must all be built + maintained. Cross-GPU detection is **not** byte-identical ? parity asserted at the window level. The `byo\_worker` tier carries a real ops/support burden (deferred, power-user only).

## 9. user (2026-07-07T14:51:10.750Z)

=== deploy/cloudrun/README.md ===

## Cloud Run GPU serving ? build / deploy / run (COMPUTE\_DECOUPLED\_SERVING M6)

The cloud-serving compute path: a control plane (`CloudRunCompute`, `backend/compute/cloud\_run.py`) dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then **bucket-polls** a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.

> **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real build + push +** run on an L4 is the **M7** owner step, within the **\$100/?10k** budget (D17) ? confirm cmd + hourly cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does **not** build this image.

## 0. Prerequisites (owner)

- A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs enabled.
- GPU quota for L4 (`GPUS\_ALL\_REGIONS` ? 1) in the chosen region (asia-south1, else Singapore ? see `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
- The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):  
`gs://<bucket>/models/wasb\_badminton\_best.pth.tar`.

## 1. Build + push the image (Artifact Registry)

```
``bash
```

```
PROJECT=<gcp-project>; REPO=rally
```

## L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai)

```
REGION=asia-southeast1
```

```
gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
```

```
2>/dev/null || true
```

## `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:

```
gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
...
```

\*(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per D17.)\*

## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)

The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region bucket** mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step needed).

```
``bash
```

```
gcloud run jobs create rally-worker \
```

```
 --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
 --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
 --cpu 8 --memory 32Gi \
 --max-retries 0 --task-timeout 3600 \
 --set-env-vars WASB_WEIGHTS=gs/models/wasb_badminton_best.pth.tar \
 --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
```

```
--add-volume-mount volume=serving,mount-path=/gcs
...
- **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it also halves the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive Y/n which, in a non-interactive shell, **hangs forever with zero output and nothing created**. An empty `gcloud` that "did nothing" ? suspect a hidden prompt.
- 8 vCPU / 32 GiB: x264 encode is CPU-side, so CPU headroom keeps the paid GPU busy. `/tmp` is **RAM-backed** ? the pulled input video lives in memory, so size --memory` for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of tmpfs).
```

## 2a. GPU-resident WASB decode (#506/#507) ? the current serving default

The image ships the **\*\*py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec==2.1.0\*\*** WASB env with ``libnvcuvid`/`libnvidia-encode`` **\*\*baked in\*\*** (multi-stage layer from the matching NVIDIA driver ? Cloud Run injects the host driver, not these userspace codec libs).  
``indexing.wasb.decode_backend`` selects the decode path:

- ``cpu`` (model default) ? the historical cv2 decode; **\*\*byte-unchanged\*\*** by this work.
- ``nvdec_resident`` ? NVDEC decode + cv2-faithful affine ``grid_sample`` resize + GPU normalize, all on-GPU. **\*\*The cloud-serving preset default\*\*** (validated: offline golden gate @ SERVED F1 0.574 ? baseline 0.582, AND on the live L4 the served GX010128 trajectory reproduced the VM run ? 26 479 ? 26 481 visible). CUDA-only (config-validator-enforced).

**\*\*How the knob reaches the worker.\*\*** The worker Job runs with NO ``DEPLOYMENT_PROFILE``, so it does **\*\*not\*\*** apply the cloud-serving preset itself. The control plane (which *\*does\** have the profile) resolves the config and **\*\*forwards\*\*** ``indexing.wasb.{decode_backend,batch_size,prefetch_batches}`` to the worker as `--set`` overrides (``orchestration.py``, alongside ``skip_ai_handoff``). So the **\*\*preset is the single source of truth\*\*** ? changing ``presets.py`` + redeploying the control plane changes serving; no per-Job knob needed.

**\*\*Override / rollback lever (no redeploy).\*\*** The native runner reads ``WASB_DECODE_BACKEND`` FIRST, so a Job env var wins over the forwarded `--set``:

```
```bash
```

force nvdec (or pin cpu) on the Job regardless of the forwarded config; instant rollback

```
gcloud run jobs update rally-worker --region $REGION --update-env-vars
WASB_DECODE_BACKEND=nvdec_resident
gcloud run jobs update rally-worker --region $REGION --remove-env-vars WASB_DECODE_BACKEND # ?
forwarded/preset default
```

full revert: pin the prior image by digest

```
gcloud run jobs update rally-worker --region $REGION --image $REGION-
docker.pkg.dev/$PROJECT/$REPO/rally-worker@sha256:<prev>
```
```

? Forwarding ``decode_backend=nvdec_resident`` requires a worker image that understands `--decode-backend`` (? #507). Roll the **\*\*worker image and the control plane together\*\*** ? a new control plane forwarding nvdec to an old worker image `argparse-fails` (rc 2).

**\*\*Post-deploy smoke.\*\*** After a cutover, confirm the real L4 path with ``deploy/cloudrun/post_deploy_smoke.ps1`` (run from PowerShell ? see \$Gotchas on MSYS arg-mangling):

```
```powershell
./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 -Decode cpu # image sanity (fast)
./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-corpus/videos/GX010128.MP4
-Decode env # nvdec via preset/env (hard clip)
```
```

## 3. Wire the dispatcher (control plane)

The control plane runs with the **\*\*cloud-serving\*\*** profile (``DEPLOYMENT_PROFILE=cloud-serving`` ?

```
`compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute` from env:
```

```
```bash
```

```
export DEPLOYMENT_PROFILE=cloud-serving
```

```
export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION GOOGLE_CLOUD_PROJECT=$PROJECT
```

A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job in running in-process; the container runs the SAME worker INLINE (compute_target for

```
python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri gs://<bucket>
--segmenter wasb_hybrid
...`
```

****The AI handoff needs its key IN THE JOB env**** (`GEMINI\_API\_KEY` ? mount a Secret Manager secret on the job), or run fully-local with `--set indexing.skip\_ai\_handoff=true` (candidate windows emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to a silent 0-rally "success" ****after**** the paid GPU pass; the cloud-serving startup checks now fail this at boot (`AI\_HANDOFF\_KEY\_MISSING`, rc 2) before the video is even pulled.

3a. Control-plane SERVICE deploy ? required flags for reliable async jobs

The API server (`/api/process`, `/api/compile` ? #329) runs its ****job worker IN-PROCESS****: the request returns `202` and a background thread drains the job (GPU detection dispatches to the Job;

the reel stitch runs on the control plane itself). On Cloud Run that background thread only makes

progress while the instance has CPU, so a scale-to-zero ****service**** deploy MUST set:

****The canonical deploy path is the committed manifest generator**** (#473) ? it bakes the cloud-serving config.json into a startup wrapper, sets every flag below, pins `maxScale=1` (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD\_RUN\_JOB` (a reserved runtime env name that gets a manifest rejected; the code default is already right):

```
```bash
```

```
python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
```

```
gcloud run services replace service.yaml --region $REGION # from PowerShell on Windows
```

```
...`
```

`--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with `on` before testers touch the service again. The flag-based equivalent (for reference ? a manifest round-trips the bash wrapper's metacharacters, flags don't on Windows):

```
```bash
```

```
gcloud run deploy rally-control-plane --image <image> --region $REGION \
```

```
--no-cpu-throttling \           # CPU stays allocated between requests ? the drain thread
finishes
```

```
--min-instances 1 \           # one always-warm instance drains reliably (cheap CPU; NOT the D7
GPU pool)
```

```
--cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
```

```
--set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
...`
```

Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then ****stalls**** ? the reel never finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT contradict

D7 (that was the ~\$500/mo warm ****GPU**** pool); this is a ~1-vCPU CPU instance. Defense-in-depth is in

the code: `Database.recover\_stale\_jobs()` ****re-queues**** a compile interrupted by a restart (idempotent

stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on the next drain.

Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to the CF proxy

(M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so flipping `security.edge\_auth\_enabled` on needs no derived image ? the M9 Cf-Access JWT verify works out of

the box.

4. M7 acceptance (the owner runlog)

Capture a ``DEPLOY_REPORT_cloud-serving_<date>.md`` under ``docs/COMPUTE_DECOUPLED_SERVING/runlogs/`` (mirrors the truly-local template): the exact commands, rally counts, ``gpu_active_seconds`/`wall_seconds`/`bytes_out``, a reel smoke-test (``ffmpeg -f null -`` exit 0 + a 5-frame contact sheet), gotchas, and the `**$0 teardown audit**`.

Gotchas

- The dispatched container `**must run inline**` ? ``CloudRunCompute`` forces ``compute_target=local_gpu`` + ``indexing.detector_impl=native`` in the per-execution args so the Job never re-dispatches itself.
- Completion is a `**bucket marker**` (``<out_uri>/_dispatch/<token>.done``), not the Cloud Run API status ? so it reuses the storage layer + ``execution_policy.poll_until`` and survives a control-plane restart.
- A `**failed**` Job (validation rc 2 / segmenter rc 3) now writes a `**failure marker**` carrying the real non-zero rc, so the dispatcher's poll terminates `**FAST + accurately**` (no 30-min hang on a bad video). Only a `**hung/crashed**` container (no marker at all) makes the bounded poll time out (``ExecutionPolicy.max_wait_sec``); the worker CLI maps that to a clean `**rc 4**`, and the API surfaces it as a ``cloud dispatch error`` job failure (not a traceback).
- `libx264` encode (parity, D16) + NVDEC decode-only; the image keeps `ffmpeg`'s default `libx264`.
- `**GOOGLE_CLOUD_PROJECT`` is required\*\* for the real client ? ``GoogleCloudRunJobClient.run_job`` fails fast with a clear ``RuntimeError`` if it is unset/empty (it must resolve a real ``job_path``; an unset project would otherwise build a bogus ``projects/None/...`` name). Export it (step 3) before dispatching. The guard runs before the ``google-cloud-run`` import, so it is exercised in CI even though the SDK isn't a CI/test dependency.
- `**Hand-rolled `jobs execute --args` with `gs://` URIs:**` `gcloud`'s default comma-split plus its alternate-delimiter syntax both bite ? repeated ``--set`` tokens trip the comma parser, and a ``^:`` delimiter splits the ``:`` inside ``gs://``. Use a delimiter that appears nowhere in the args, e.g. ``--args='^@@^~v@@infer@@--video-uri=gs://?'``. (The dispatcher path doesn't have this problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must include the dispatcher's two forced ``--set`s``: ``deployment.compute_target=local_gpu``, ``indexing.detector_impl=native``.
- `**Git Bash (MSYS) on Windows mangles Unix-looking values**` before `gcloud` sees them: ``--add-volume-mount ?mount-path=/gcs`` became ``C:/Program Files/Git/gcs``, and ``--command=/bin/bash`` became ``C:/Program Files/Git/usr/bin/bash`` (the container then exec-fails). Run `gcloud` commands with such flags from PowerShell (``MSYS_NO_PATHCONV=1`` breaks `gcloud`'s own wrapper). PowerShell has its own trap: quote any flag value containing commas.
- `**First execution after an image push**` spends minutes in "Container image import in progress" (visible in ``executions describe status.conditions``, NOT in logs) ? that's progress, not a hang; later executions reuse the imported image.

10. assistant (2026-07-07T14:53:06.371Z)

The finding hinges on comparing ``--skip-validation`` to the ``decode_backend`` forward. Let me verify the exact forwarding mechanisms and the worker's `argparse`/error handling.

11. user (2026-07-07T14:53:08.371Z)

```
540         logger.info(  
541             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",  
542             req.video_path,
```

```

543         out_root,
544     )
545     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549     overrides = []
550     idx_cfg = getattr(cfg, "indexing", None)
551     sk = getattr(idx_cfg, "skip_ai_handoff", None)
552     if sk is not None:
553         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
554     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
555     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
556     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
557     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
558     # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
559     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
560     wasb_cfg = getattr(idx_cfg, "wasb", None)
561     if wasb_cfg is not None:
562         decode_backend = getattr(wasb_cfg, "decode_backend", None)
563         if decode_backend:
564             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565         batch_size = getattr(wasb_cfg, "batch_size", None)
566         if batch_size:
567             overrides.append(f"indexing.wasb.batch_size={batch_size}")
568         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569         if prefetch:
570             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571     # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
572     # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
573     # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
574     # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
575     # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
576     # then worker-rejected at its default 20.0 after a successful GPU dispatch).
577     spec = ComputeJobSpec(
578         video_uri=req.video_path,
579         out_uri=out_root,
580         segmenter=seg_name,
581         config_overrides=tuple(overrides),
582         skip_validation=True,
583     )
584     try:
585         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
586         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
587         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
588         # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
589         result = compute.run_infer(
590             spec,

```

```

591         on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
592     )
593     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
594         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)

```

12. user (2026-07-07T14:53:11.287Z)

```

=== worker main() / arg handling ===
def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
    _setup_logging(getattr(args, "verbose", False))
    return args.func(args)

if __name__ == "__main__":
    sys.exit(main())
=== presets.py 55-75 ===
# <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
# CPU-feed-bound. Serving opts into the measured fast values; local/WSL unchanged.
"indexing": {
    "detector_impl": "native",
    "skip_ai_handoff": False,
    # GPU-resident NVDEC decode (#506/#507): validated on a real L4 (golden gate PASS,
    # F1 0.574 ? baseline 0.582 @ SERVED) AND on the live Cloud Run L4 (the served
    # trajectory reproduced the VM run ? 26479 ? 26481 visible on GX010128), then rolled
    # out + canaried (DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07). This
makes
    # nvdec the code-level serving default so it survives an image rebuild; the worker's
    # WASB_DECODE_BACKEND env was the reversible rollout lever. cuda-only (validator-
enforced).
    "wasb": {
        "batch_size": 64,
        "prefetch_batches": 4,
        "decode_backend": "nvdec_resident",
    },
},
"storage": {"backend": "gcs"},
},
"cpu-mock": {
    "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},

```

13. assistant (2026-07-07T14:53:56.148Z)

I've now traced both the mechanism and its already-accepted precedent. Let me confirm the one remaining fact: whether the decode_backend forward is truly ~100% on the serving path (which determines if skip_validation's blast radius is uniquely worse, as the finding claims).

I have what I need. Analysis:

****The mechanism is real but not a code defect, and the finding's key differentiator is factually wrong.****

What I verified in the real code:

- `backend/worker/\_\_main\_\_.py::main()` is `args = build\_parser().parse\_args(argv)` with no guard ? an unknown `--skip-validation` on an old worker image would indeed raise `SystemExit(2)`. So the raw argparse-fail mechanism is genuine.
- `backend/compute/cloud\_run.py` appends `--skip-validation` (hard flag) and `backend/orchestration.py:582` sets `skip\_validation=True` unconditionally. Confirmed.

Why the finding is REFUTED as a major/novel rollout defect:

1. ****This is the identical, already-accepted, already-documented hazard ? not a new one.****
`deploy/cloudrun/README.md:76-78` already warns: forwarding a new knob to an old worker image
"argparse-fails (rc 2). Roll the worker image and the control plane together." Control-plane and worker are the ****same image****, just separate deploy targets. The documented deploy discipline

(roll together) already governs exactly this class of change. Adding one more forwarded flag sits inside that same accepted risk envelope.

2. ****The finding's core differentiator ? "worse blast radius than decode_backend because skip_validation is forwarded on 100% of dispatches unconditionally" ? is factually false.****
`backend/config/presets.py:69` makes `decode\_backend: nvdec\_resident` the cloud-serving ****preset default****, and `orchestration.py:562-564` forwards it whenever set. So decode_backend is ***also*** forwarded on ~100% of cloud-serving dispatches. Both flags have the same serving blast radius. The claimed asymmetry doesn't exist.

3. ****The "WASB_DECODE_BACKEND escape hatch" the finding cites as decode_backend's advantage does not mitigate the skew**** ? a truly-old worker image that doesn't understand the knob fails regardless of that env var (it only overrides the ***forwarded value*** on a worker that already supports it). So it's not a relevant differentiator either.

4. ****`--segmenter` is precedent for hard, unconditionally-forwarded flags.**** `cloud\_run.py` already forwards ****`--segmenter`** as a hard argv flag on every dispatch; the same roll-ordering constraint has always applied. skip_validation introduces no new class of risk.

5. The scenario requires an operator to violate the documented "roll together" step, and produces a ****loud, fast, immediately-diagnosable**** rc 2 that is reversible by the very remediation already in the runbook (roll the worker image). No code change corrects a deploy-ordering discipline.

The only legitimate kernel: the `deploy/cloudrun/README.md:76-78` roll-together note enumerates only ****`--decode-backend`** by name and wasn't extended to mention ****`--skip-validation`**. That is a cheap doc-hygiene nicety, not a major correctness/rollout defect in the code.

14. user (2026-07-07T14:54:11.396Z)

Structured output provided successfully